Tx Circuit

개요

Tx Circuit은 블록 내 모든 트랜잭션의 유효성을 검증하는 회로입니다. 이 회로는 트랜잭션의 서명, RLP 인코딩, 가스 비용, 그리고 트랜잭션 타입별 특성을 검증하며, Tx Table을 통해 다른 회로들과 트랜잭션 데이터를 공유합니다.

주요 역할:

- 트랜잭션 서명 검증 (ECDSA)
- RLP 인코딩 정확성 검증
- 트랜잭션 해시 계산 및 검증
- CallData 가스 비용 계산
- Access List 처리 (EIP-2930, EIP-1559)
- 트랜잭션 타입별 필드 검증
- Tx Table을 통한 데이터 제공

파일 위치:

- 메인 파일: `zkevm-circuits/src/tx_circuit.rs`
- Witness 생성: `zkevm-circuits/src/witness/tx.rs`
- 테이블 정의: `zkevm-circuits/src/table.rs`

트랜잭션 타입

Scroll ZKEVM은 다양한 이더리움 트랜잭션 타입을 지원합니다.

TxType 열거형

```
pub enum TxType {
PreEip155 = 0, // Legacy 트랜잭션 (EIP-155 이전)
Eip155 = 1, // EIP-155 트랜잭션 (Chain ID 포함)
Eip2930 = 2, // EIP-2930 (Access List 포함)
Eip1559 = 3, // EIP-1559 (Max Fee, Priority Fee)
L1Msg = 4, // L1에서 L2로의 메시지
}
```

트랜잭션 타입별 특징

PreEip155 (Legacy)

- Chain ID 없음
- 서명: (v, r, s) - v는 27 또는 28
- RLP 인코딩: `[nonce, gasPrice, gas, to, value, data, v, r, s]`

Eip155

- Chain ID 포함 (replay attack 방지)
- 서명: v = chain_id * 2 + 35 + recovery_id
- 서명 해시: `keccak256(rlp([nonce, gasPrice, gas, to, value, data, chain_id, 0, 0]))`
- 트랜잭션 해시: `keccak256(rlp([nonce, gasPrice, gas, to, value, data, v, r, s]))`

Eip2930

- Access List 포함 (스토리지 접근 최적화)
- 타입 바이트: `0x01`
- RLP 인코딩: `0x01 || rlp([chain_id, nonce, gasPrice, gas, to, value, data, accessList, v, r, s])`
- Access List 구조: `[[address, [storageKey1, storageKey2, ...]], ...]`

Eip1559

- 동적 가스 가격 (Base Fee + Priority Fee)
- 타입 바이트: `0x02`
- 필드: `maxFeePerGas` , `maxPriorityFeePerGas`
- RLP 인코딩: `0x02 || rlp([chain_id, nonce, maxPriorityFeePerGas, maxFeePerGas, gas, to, value, data, accessList, v, r, s])`

L1Msg

- L1에서 L2로 전송되는 메시지
- 서명 없음 (L1에서 이미 검증됨)
- 특별한 해시 계산 방식
- `tx_data_gas_cost = 0` (L1에서 이미 지불)

TxFieldTag 열거형

Tx Table의 각 행은 트랜잭션의 특정 필드를 나타냅니다.

```
pub  enum TxFieldTag {

Null = 0, // 사용되지 않는 태그

CallData, // 트랜잭션 데이터 (동적)
```

```
Nonce, // 발신자 nonce

GasPrice, // 가스 가격

Gas, // 가스 한도

CallerAddress, // 발신자 주소

CalleeAddress, // 수신자 주소

IsCreate, // 컨트랙트 생성 여부

Value, // 전송 이더 양

CallDataRLC, // CallData의 RLC

CallDataLength, // CallData 길이

CallDataGasCost, // CallData 가스 비용

TxDataGasCost, // RLP 인코딩 가스 비용

ChainID, // Chain ID

SigV, // 서명 V 값

SigR, // 서명 R 값

SigS, // 서명 S 값

TxSignLength, // 서명용 RLP 길이

TxSignRLC, // 서명용 RLP의 RLC

TxSignHash, // 서명 해시

TxHashLength, // 트랜잭션 해시용 RLP 길이

TxHashRLC, // 트랜잭션 해시용 RLP의 RLC

TxHash, // 트랜잭션 해시

TxType, // 트랜잭션 타입

AccessListAddress, // Access List 주소 (동적)
```

```
AccessListStorageKey, // Access List 스토리지 키 (동적)

AccessListAddressesLen, // Access List 주소 개수

AccessListStorageKeysLen, // Access List 스토리지 키 총 개수

AccessListRLC, // Access List의 RLC

BlockNumber, // 블록 번호

MaxPriorityFeePerGas, // 최대 우선순위 수수료 (EIP-1559)

MaxFeePerGas, // 최대 수수료 (EIP-1559)

}
```

필드 설명

고정 필드 (Fixed Fields)

각 트랜잭션은 28개의 고정 행을 가집니다 (`TX_LEN = 28`):

- Nonce부터 BlockNumber까지의 필드
- 모든 트랜잭션에 대해 동일한 순서로 배치
- 트랜잭션 타입에 따라 일부 필드는 0 또는 Null 값

동적 필드 (Dynamic Fields)

- **CallData**: 가변 길이 데이터, 각 바이트가 별도 행
- **AccessListAddress**: Access List의 각 주소
- **AccessListStorageKey**: 각 주소의 스토리지 키들

회로 구조

TxCircuitConfig

```rust
pub struct TxCircuitConfig<F: Field> {
// 고정 컬럼
q_enable: Column<Fixed>, // 활성화 플래그
q_first: Column<Fixed>, // 첫 행 표시
q_dynamic_first: Column<Fixed>, // 동적 섹션 시작
q_dynamic_last: Column<Fixed>, // 동적 섹션 종료
// Tx Table
tx_table: TxTable, // tx_id, tag, index, value
// 트랜잭션 타입 및 메타데이터
tx_type: Column<Advice>, // 트랜잭션 타입
tx_type_bits: BinaryNumberConfig<TxType, 3>, // 타입 비트 분해
rlp_tag: Column<Advice>, // RLP 테이블 태그
is_none: Column<Advice>, // 값이 0x80 (빈 값) 여부
tx_value_length: Column<Advice>, // 값의 길이
tx_value_rlc: Column<Advice>, // 값의 RLC
// Tag 비트 분해
tx_tag_bits: BinaryNumberConfig<TxFieldTag, 5>,
// IsZero 가젯
tx_id_is_zero: IsZeroConfig<F>, // tx_id == 0 검증
value_is_zero: IsZeroConfig<F>, // value == 0 검증
tx_id_unchanged: IsEqualConfig<F>, // tx_id 변화 검증
// 도수 감소용 boolean 컬럼
```

```
is_tag_block_num: Column<Advice>,
is_calldata: Column<Advice>,
is_caller_address: Column<Advice>,
is_row_hash_rlc: Column<Advice>,
is_l1_msg: Column<Advice>,
is_eip2930: Column<Advice>,
is_eip1559: Column<Advice>,
is_chain_id: Column<Advice>,
is_tx_id_zero: Column<Advice>,
// CallData 누적
is_final: Column<Advice>, // CallData 마지막 바이트
calldata_gas_cost_acc: Column<Advice>, // 가스 비용 누적
section_rlc: Column<Advice>, // CallData/AccessList RLC
calldata_byte: Column<Advice>, // CallData 바이트 값
// 블록 번호 검증
is_padding_tx: Column<Advice>, // 패딩 트랜잭션 여부
tx_id_cmp_cum_num_txs: ComparatorConfig<F, 2>,
cum_num_txs: Column<Advice>, // 누적 트랜잭션 수
num_txs: Column<Advice>, // 블록당 트랜잭션 수
// 서명 검증
sv_address: Column<Advice>, // 복구된 주소
// Access List
al_idx: Column<Advice>, // Access List 주소 인덱스
```

```
sk_idx: Column<Advice>, // 스토리지 키 인덱스

sks_acc: Column<Advice>, // 스토리지 키 누적

is_access_list: Column<Advice>,

is_access_list_address: Column<Advice>,

is_access_list_storage_key: Column<Advice>,

field_rlc: Column<Advice>, // RLP FSM 필드 RLC

// Chunk bytes 누적 (증명 집계용)

is_chunk_bytes: Column<Advice>,

chunk_bytes_len: Column<Advice>,

chunk_txbytes_rlc: Column<Advice>,

chunk_txbytes_len_acc: Column<Advice>,

pow_of_rand: Column<Advice>,

// 외부 테이블

sig_table: SigTable, // 서명 검증 테이블

block_table: BlockTable, // 블록 정보 테이블

rlp_table: RlpTable, // RLP 인코딩 테이블

keccak_table: KeccakTable, // Keccak 해시 테이블

u8_table: U8Table, // 8비트 룩업

u16_table: U16Table, // 16비트 룩업

// ROM 테이블

tx_rom_table: TxRomTable, // Tag 전환 규칙

}
```

TxRomTable (Tag 전환 규칙)

TxRomTable은 Tx Table의 tag 전환이 올바른지 검증하는 고정 테이블입니다.

```
pub struct TxRomTable {
pub tag: Column<Fixed>, // 현재 tag
pub tag_next: Column<Fixed>, // 다음 tag
pub is_tx_id_unchanged: Column<Fixed>, // tx_id 유지 여부
pub is_final: Column<Fixed>, // 동적 섹션 완료
pub is_next_dynamic_first: Column<Fixed>, // 다음이 동적 섹션 시작
}
```

전환 시나리오 예시:

```
// 고정 섹션 내부 전환 (tx_id 유지)
(Nonce, GasPrice, 1, 1, 0)
(GasPrice, Gas, 1, 1, 0)
(Gas, CallerAddress, 1, 1, 0)
...

// 고정 섹션에서 동적 섹션으로 전환
(BlockNumber, CallData, 1, 1, 1) // CallData로 전환
(BlockNumber, AccessListAddress, 1, 1, 1) // AccessList로 전환

// 동적 섹션 내부
```

```
(CallData, CallData, 1, 0, 0) // CallData 계속
(CallData, CallData, 0, 1, 0) // CallData 종료, 다음 tx
(AccessListAddress, AccessListStorageKey, 1, 0, 0) // 스토리지 키로
(AccessListStorageKey, AccessListAddress, 1, 0, 0) // 다음 주소로

// 트랜잭션 간 전환
(BlockNumber, Nonce, 0, 1, 0) // 다음 tx 시작
```

서명 검증 프로세스 (ECDSA)

서명 데이터 구조

```
pub struct SignData {
pub signature: (Fq, Fq, u8), // (r, s, v)
pub pk: Secp256k1Affine, // 공개키
pub msg: Vec<u8>, // 서명할 메시지 (RLP)
pub msg_hash: Fq, // 메시지 해시
}
```

서명 검증 단계

1. 서명 해시 계산

트랜잭션 타입에 따라 서명할 데이터가 다릅니다:

PreEip155:

```
sign_data = rlp([nonce, gasPrice, gas, to, value, data])

sign_hash = keccak256(sign_data)
```

Eip155:

```
sign_data = rlp([nonce, gasPrice, gas, to, value, data, chain_id, 0, 0])

sign_hash = keccak256(sign_data)
```

Eip2930:

```
sign_data = 0x01 || rlp([chain_id, nonce, gasPrice, gas, to, value, data, access

sign_hash = keccak256(sign_data)
```

Eip1559:

```
sign_data = 0x02 || rlp([chain_id, nonce, maxPriorityFeePerGas, maxFeePerGas,

gas, to, value, data, accessList])

sign_hash = keccak256(sign_data)
```

2. 공개키 복구 (ecrecover)

ECDSA 서명 (r, s, v)로부터 공개키를 복구합니다:

```
pub fn recover_pk2(v: u8, r: &Word, s: &Word, msg_hash: &[u8; 32])

-> Result<Secp256k1Affine, Error>
```

```
{
// v에서 recovery_id 추출
let recovery_id = v % 2;
// secp256k1 서명 복구
let sig = Signature::from_compact(&[r, s])?;
let pk = sig.recover(msg_hash, recovery_id)?;
Ok(pk)
}
```

3. 주소 계산

공개키로부터 이더리움 주소를 계산합니다:

```
// 공개키를 바이트로 변환 (압축되지 않은 형식, 64바이트)
let pk_bytes = pk.to_bytes_uncompressed();

// Keccak256 해시 계산
let hash = keccak256(&pk_bytes[1..]); // 첫 바이트(0x04) 제외

// 마지막 20바이트가 주소
let address = Address::from_slice(&hash[12..]);
```

4. 서명 검증

Sig Circuit과의 룩업을 통해 서명을 검증합니다:

```
meta.lookup_any("sig_table_lookup", |meta| {
let  enable = is_chain_id(meta);
vec![
(enable.clone() * tx_id, sig_table.tx_id),
(enable.clone() * sig_v, sig_table.sig_v),
(enable.clone() * sig_r, sig_table.sig_r),
(enable.clone() * sig_s, sig_table.sig_s),
(enable.clone() * sign_hash, sig_table.msg_hash),
(enable * sv_address, sig_table.recovered_addr),
]
});
```

CallerAddress 검증

복구된 주소(`sv_address`)가 CallerAddress와 일치하는지 검증합니다:

```
cb.condition(is_caller_address(meta), |cb| {
cb.require_equal(
"caller_address == sv_address",
tx_table.value,
sv_address,
);
});
```

예외: CallerAddress가 0이면 서명 검증을 건너뜁니다 (패딩 트랜잭션).

RLP 인코딩 검증

Tx Circuit은 RLP Table과의 룩업을 통해 트랜잭션의 RLP 인코딩이 올바른지 검증합니다.

RLP 태그 매핑

각 TxFieldTag는 대응하는 RLP 태그를 가집니다:

```
let  rlp_tag_map: Vec<(Expression<F>, RlpTag)> = vec![
(is_nonce(meta), Tag::Nonce.into()),
(is_gas_price(meta), Tag::GasPrice.into()),
(is_gas(meta), Tag::Gas.into()),
(is_to(meta), Tag::To.into()),
(is_value(meta), Tag::Value.into()),
(is_data_rlc(meta), Tag::Data.into()),
(is_sig_v(meta), Tag::SigV.into()),
(is_sig_r(meta), Tag::SigR.into()),
(is_sig_s(meta), Tag::SigS.into()),
(is_sign_length(meta), RlpTag::Len),
(is_sign_rlc(meta), RlpTag::RLC),
(is_hash_length(meta), RlpTag::Len),
(is_hash_rlc(meta), RlpTag::RLC),
(is_caller_addr(meta), Tag::Sender.into()),
(is_tx_gas_cost(meta), RlpTag::GasCost),
```

```
(is_access_list_address(meta), Tag::AccessListAddress.into()),
(is_access_list_storage_key(meta), Tag::AccessListStorageKey.into()),
(is_max_fee_per_gas(meta), Tag::MaxFeePerGas.into()),
(is_max_priority_fee_per_gas(meta), Tag::MaxPriorityFeePerGas.into()),
(is_chain_id(meta), Tag::ChainId.into()),
// Null 태그들
(is_null(meta), RlpTag::Null),
(is_create(meta), RlpTag::Null),
(is_data_length(meta), RlpTag::Null),
(is_data_gas_cost(meta), RlpTag::Null),
(is_sign_hash(meta), RlpTag::Null),
(is_hash(meta), RlpTag::Null),
(is_data(meta), RlpTag::Null),
(is_block_num(meta), RlpTag::Null),
(is_tx_type(meta), RlpTag::Null),
(is_access_list_addresses_len(meta), RlpTag::Null),
(is_access_list_storage_keys_len(meta), RlpTag::Null),
(is_access_list_rlc(meta), RlpTag::RLC),
];
```

RLP 테이블 룩업

서명용 RLP (TxSign)

서명 해시 계산에 사용되는 RLP 인코딩을 검증합니다:

```
meta.lookup_any("rlp_sign_tag_lookup", |meta| {
let  enable = is_tag_in_tx_sign(meta); // 서명에 포함되는 태그들
vec![
(enable.clone() * tx_id, rlp_table.tx_id),
(enable.clone() * rlp_tag, rlp_table.tag),
(enable.clone() * is_none, rlp_table.is_none),
(enable.clone() * tx_value_length, rlp_table.value_length),
(enable * tx_value_rlc, rlp_table.value_rlc),
]
});
```

포함되는 태그 (트랜잭션 타입에 따라 다름):

- 공통: Nonce, Gas, To, Value, Data
- Eip155/Eip2930/Eip1559: ChainId
- Eip1559: MaxFeePerGas, MaxPriorityFeePerGas
- PreEip155: GasPrice (ChainId 없음)

트랜잭션 해시용 RLP (TxHash)

트랜잭션 해시 계산에 사용되는 RLP 인코딩을 검증합니다:

```
meta.lookup_any("rlp_hash_tag_lookup", |meta| {
let  enable = is_tag_in_tx_hash(meta); // 해시에 포함되는 태그들
vec![
```

```
(enable.clone() * tx_id, rlp_table.tx_id),
(enable.clone() * rlp_tag, rlp_table.tag),
(enable.clone() * is_none, rlp_table.is_none),
(enable.clone() * tx_value_length, rlp_table.value_length),
(enable * tx_value_rlc, rlp_table.value_rlc),
]
});
```

포함되는 태그:

- 서명용 태그 + 서명 필드 (SigV, SigR, SigS)
- TxDataGasCost (RLP 인코딩 가스 비용)

L1 메시지 RLP

L1Msg 타입은 별도의 RLP 형식을 사용합니다:

```
meta.lookup_any("l1_msg_hash_lookup", |meta| {
let  enable = is_tag_in_l1_msg_hash(meta);
vec![
(enable.clone() * tx_id, rlp_table.tx_id),
(enable.clone() * rlp_tag, rlp_table.tag),
(enable.clone() * is_none, rlp_table.is_none),
(enable.clone() * tx_value_length, rlp_table.value_length),
(enable * tx_value_rlc, rlp_table.value_rlc),
]
```

```
});
```

포함되는 태그: Nonce, Gas, To, Value, Data, CallerAddress

is_none 처리

RLP에서 빈 값은 `0x80` 으로 인코딩됩니다. `is_none` 플래그는 이를 표시합니다.

```
// is_none == true이면 value == 0
cb.condition(is_none_expr, |cb| {
cb.require_equal(
"is_none => value == 0",
tx_table.value,
0.expr(),
);
});


// CallDataRLC가 none이면 CallDataLength와 CallDataGasCost도 0
cb.condition(and::expr([is_data_rlc(meta), is_none_expr]), |cb| {
cb.require_equal("CallDataLength == 0",
tx_table.value(Rotation::next()), 0.expr());
cb.require_equal("CallDataGasCost == 0",
tx_table.value(Rotation(2)), 0.expr());
});
```

CallData 처리 및 가스 비용 계산

CallData 구조

CallData는 동적 섹션에 저장되며, 각 바이트가 별도의 행을 차지합니다.

Tx Table 구조:

```
| tx_id | tag | index | value |
|-------|----------|-------|-------|
| 1 | CallData | 0 | 0x60 |
| 1 | CallData | 1 | 0x00 |
| 1 | CallData | 2 | 0x42 |
| ... | ... | ... | ... |
```

가스 비용 계산

CallData의 가스 비용은 바이트 값에 따라 다릅니다:

- 0 바이트: 4 gas
- 비-0 바이트: 16 gas

```
// 각 CallData 바이트에 대해
let  gas_cost = if  byte == 0 { 4 } else { 16 };
calldata_gas_cost_acc = calldata_gas_cost_acc_prev + gas_cost;
```

is_final 플래그

`is_final` 은 CallData의 마지막 바이트를 표시합니다.

```
// CallData의 마지막 바이트
cb.condition(and::expr([is_calldata(meta), is_final(meta)]), |cb| {
// 다음 행은 CallData가 아니거나 다른 tx
cb.require_zero(
"next is not calldata or different tx",
and::expr([
is_calldata(Rotation::next()),
tx_id_unchanged.is_equal_expression,
]),
);
});
```

CallData RLC 검증

CallData의 RLC는 고정 섹션의 `CallDataRLC` 필드와 일치해야 합니다:

```
// 마지막 CallData 바이트에서
cb.condition(and::expr([is_calldata(meta), is_final(meta)]), |cb| {
cb.require_equal(
"section_rlc matches CallDataRLC",
section_rlc,
calldata_rlc_from_fixed_section,
);
```

```
});
```

CallData 길이 검증

```
// CallDataLength 행에서
cb.condition(is_data_length(meta), |cb| {
// CallDataLength가 0이 아니면 동적 섹션에 CallData 존재
cb.condition(not::expr(value_is_zero.expr()), |cb| {
// 동적 섹션 룩업 활성화
cb.require_equal(
"lookup_condition",
lookup_conditions[&LookupCondition::TxCalldata],
1.expr(),
);
});
});
```

Access List 처리 (EIP-2930, EIP-1559)

Access List는 트랜잭션이 접근할 주소와 스토리지 키를 미리 선언하여 가스 비용을 최적화합니다.

Access List 구조

```
pub struct AccessList(pub Vec<AccessListItem>);
```

```rust
pub struct AccessListItem {
pub address: Address,
pub storage_keys: Vec<H256>,
}
```

Tx Table에서의 표현

고정 섹션:

```
| tag | value |
|----------------------------|-------|
| AccessListAddressesLen | 2 | // 주소 개수
| AccessListStorageKeysLen | 5 | // 총 스토리지 키 개수
| AccessListRLC | ... | // Access List RLC
```

동적 섹션:

```
| tx_id | tag | index | value | access_list_address |
|-------|-----------------------|-------|------------|---------------------|
| 1 | AccessListAddress | 1 | addr1 | addr1 |
| 1 | AccessListStorageKey | 1 | key1 | addr1 |
| 1 | AccessListStorageKey | 2 | key2 | addr1 |
| 1 | AccessListAddress | 2 | addr2 | addr2 |
| 1 | AccessListStorageKey | 1 | key3 | addr2 |
```

```
| 1 | AccessListStorageKey | 2 | key4 | addr2 |

| 1 | AccessListStorageKey | 3 | key5 | addr2 |
```

인덱스 관리

al_idx (Address Index)

각 주소에 대해 1부터 시작하는 인덱스:

```
// AccessListAddress 행에서
al_idx = al_idx_prev + 1;


// AccessListStorageKey 행에서
al_idx = al_idx_prev; // 유지
```

sk_idx (Storage Key Index)

각 주소 내에서 스토리지 키의 인덱스:

```
// AccessListAddress 행에서
sk_idx = 0; // 리셋


// AccessListStorageKey 행에서
sk_idx = sk_idx_prev + 1;
```

sks_acc (Storage Keys Accumulator)

전체 스토리지 키의 누적 개수:

```
// AccessListStorageKey 행에서
sks_acc = sks_acc_prev + 1;
```

Access List RLC 검증

Access List의 RLC는 RLP 인코딩된 Access List의 RLC와 일치해야 합니다:

```
// 마지막 AccessList 항목에서
cb.condition(and::expr([is_access_list(meta), is_final(meta)]), |cb| {
cb.require_equal(
"section_rlc matches AccessListRLC",
section_rlc,
access_list_rlc_from_fixed_section,
);
});
```

가스 비용 계산

Access List의 가스 비용:

- 주소당: 2400 gas
- 스토리지 키당: 1900 gas

```
pub fn tx_access_list_gas_cost(access_list: &AccessList) -> u64 {
let mut cost = 0u64;
```

```
for item in &access_list.0 {
cost += 2400; // 주소
cost += 1900 * item.storage_keys.len() as u64; // 스토리지 키들
}
cost
}
```

field_rlc 검증

`field_rlc` 는 RLP FSM에서 생성된 각 필드의 RLC를 저장합니다. 이를 통해 Access List의 완전성을 보장합니다:

```
// AccessListAddress와 AccessListStorageKey의 field_rlc가
// RLP 테이블의 해당 필드 RLC와 일치하는지 검증
```

Keccak 테이블 룩업

Tx Circuit은 Keccak Table을 사용하여 트랜잭션 해시와 서명 해시를 검증합니다.

서명 해시 검증

```
meta.lookup_any("keccak_sign_hash", |meta| {
// TxSignLength 행에서 활성화 (L1Msg 제외)
let enable = and::expr([
is_sign_length(meta),
```

```
not::expr(is_l1_msg(meta)),
]);
vec![
(enable.clone(), keccak_table.is_final),
(enable.clone() * tx_sign_rlc, keccak_table.input_rlc),
(enable.clone() * tx_sign_length, keccak_table.input_len),
(enable * tx_sign_hash, keccak_table.output_rlc),
]
});
```

이 룩업은:

- 입력: `tx_sign_rlc` (서명용 RLP의 RLC), `tx_sign_length`
- 출력: `tx_sign_hash` (서명 해시)

트랜잭션 해시 검증

L2 트랜잭션 (PreEip155, Eip155, Eip2930, Eip1559)

L2 트랜잭션의 해시는 RLP Circuit에서 계산되므로, Tx Circuit에서는 0으로 설정합니다:

```
cb.condition(is_l2_hash(meta), |cb| {
cb.require_zero(
"L2 tx hash value is 0",
tx_table.value,
);
```

```
});
```

실제 해시 검증은 RLP Circuit에서 수행됩니다.

L1 메시지

L1 메시지의 해시는 Keccak Table에서 직접 검증합니다:

```
meta.lookup_any("keccak_l1_msg_hash", |meta| {
let  enable = and::expr([
is_hash_length(meta),
is_l1_msg(meta),
]);
vec![
(enable.clone(), keccak_table.is_final),
(enable.clone() * tx_hash_rlc, keccak_table.input_rlc),
(enable.clone() * tx_hash_length, keccak_table.input_len),
(enable * tx_hash, keccak_table.output_rlc),
]
});
```

Sig 테이블 룩업

Sig Circuit과의 연동을 통해 ECDSA 서명을 검증합니다.

서명 검증 룩업

```
meta.lookup_any("sig_table_lookup", |meta| {
// ChainID 행에서 활성화 (패딩 tx 제외)
let  enable = and::expr([
is_chain_id(meta),
not::expr(value_is_zero.expr(Rotation(CALLER_ADDRESS_OFFSET))),
]);
vec![
(enable.clone() * tx_id, sig_table.tx_id),
(enable.clone() * sig_v, sig_table.sig_v),
(enable.clone() * sig_r, sig_table.sig_r),
(enable.clone() * sig_s, sig_table.sig_s),
(enable.clone() * tx_sign_hash, sig_table.msg_hash),
(enable * sv_address, sig_table.recovered_addr),
]
});
```

이 룩업은:

- 입력: tx_id, sig_v, sig_r, sig_s, tx_sign_hash
- 출력: sv_address (복구된 주소)

복구된 주소 검증

```
// CallerAddress 행에서
cb.condition(is_caller_address(meta), |cb| {
```

```
// CallerAddress가 0이 아니면 (패딩 tx가 아니면)
cb.condition(not::expr(value_is_zero.expr()), |cb| {
cb.require_equal(
"caller_address == sv_address",
tx_table.value,
sv_address,
);
});
});
```

트랜잭션 타입별 제약 조건

PreEip155

```
// Chain ID 없음 (또는 0)
// v는 27 또는 28
cb.condition(tx_type_bits.value_equals(PreEip155, Rotation::cur()), |cb| {
cb.require_in_set(
"v in {27, 28}",
sig_v,
vec![27.expr(), 28.expr()],
);
```

```
});
```

Eip155

```
// Chain ID 존재
// v = chain_id * 2 + 35 + recovery_id
cb.condition(tx_type_bits.value_equals(Eip155, Rotation::cur()), |cb| {
cb.require_equal(
"v = chain_id * 2 + 35 + recovery_id",
sig_v,
chain_id * 2.expr() + 35.expr() + recovery_id,
);
});
```

Eip2930

```
// Access List 존재
// 타입 바이트: 0x01
cb.condition(is_eip2930(meta), |cb| {
// Access List 관련 필드 검증
cb.require_zero(
"AccessListAddressesLen > 0 or == 0",
// Access List가 있거나 없거나 둘 다 유효
```

```
);
});
```

Eip1559

```
// MaxFeePerGas와 MaxPriorityFeePerGas 존재
// 타입 바이트: 0x02
cb.condition(is_eip1559(meta), |cb| {
// GasPrice 대신 MaxFeePerGas 사용
cb.require_zero(
"GasPrice is not used",
gas_price,
);
// MaxPriorityFeePerGas <= MaxFeePerGas
// (이 검증은 EVM Circuit에서 수행)
});
```

L1Msg

```
// 서명 없음
// TxDataGasCost == 0
cb.condition(is_l1_msg(meta), |cb| {
cb.condition(is_tx_gas_cost(meta), |cb| {
```

```
cb.require_zero(
"tx_gas_cost == 0 for L1Msg",
tx_table.value,
);
});
// 서명 필드는 모두 0
cb.require_zero("sig_v == 0", sig_v);
cb.require_zero("sig_r == 0", sig_r);
cb.require_zero("sig_s == 0", sig_s);
});
```

블록 번호 및 트랜잭션 순서 검증

블록 번호 검증

트랜잭션의 블록 번호가 유효한지 검증합니다:

```
// BlockNumber 행에서
cb.condition(is_tag_block_num(meta), |cb| {
cb.require_equal(
"block_num = tx_table.value",
block_num,
tx_table.value,
);
```

```
});
```

블록 번호 비감소 검증

블록 번호는 감소하지 않아야 합니다 (패딩 tx 제외):

```
meta.lookup("block_num is non-decreasing", |meta| {
let is_next_tag_dynamic = sum::expr([
is_calldata(Rotation::next()),
is_access_list(Rotation::next()),
]);
let lookup_condition = and::expr([
not::expr(is_padding_tx(Rotation::next())),
not::expr(is_next_tag_dynamic),
is_tag_block_num(meta),
]);
let block_num_diff = block_num(Rotation::next()) - block_num;
vec![(lookup_condition * block_num_diff, u16_table.into())]
});
```

이 룩업은 `block_num_diff` 가 u16 범위 내의 양수임을 보장합니다.

트랜잭션 ID 검증

```
// tx_id는 1부터 시작
```

```
cb.condition(q_first(meta), |cb| {
cb.require_equal(
"tx_id == 1",
tx_table.tx_id(Rotation::next()),
1.expr(),
);
});


// 고정 섹션에서 tx_id 전환
cb.condition(tag_bits.value_equals(Nonce, Rotation::next()), |cb| {
cb.require_equal(
"tx_id increments",
tx_table.tx_id(Rotation::next()),
tx_table.tx_id + 1.expr(),
);
});
```

누적 트랜잭션 수 (num_all_txs)

블록 내 L1 메시지와 L2 트랜잭션의 총 개수를 추적합니다:

```
// 첫 트랜잭션
cb.condition(q_first(meta), |cb| {
cb.require_equal(
```

```
"num_all_txs_acc initialization",
num_all_txs_acc,
select::expr(
is_l1_msg,
// L1 msg: queue_index - total_l1_popped_before + 1
tx_nonce - total_l1_popped_before + 1.expr(),
// L2 tx: 1
1.expr(),
),
);
});


// 같은 블록 내 다음 트랜잭션
cb.condition(block_num_unchanged.expr(), |cb| {
cb.require_equal(
"num_all_txs_acc increment",
num_all_txs_acc(Rotation::next()) - num_all_txs_acc,
select::expr(
is_l1_msg(Rotation::next()),
tx_nonce(Rotation::next()) - total_l1_popped_before(Rotation::next()) + 1.expr()
1.expr(),
),
);
```

```
});
```

Chunk Bytes 누적 (증명 집계용)

Chunk bytes는 증명 집계 시 청크의 해시를 계산하는 데 사용됩니다.

is_chunk_bytes

L2 트랜잭션만 청크에 포함됩니다 (L1Msg와 패딩 tx 제외):

```
cb.require_equal(
"is_chunk_bytes",
is_chunk_bytes,
and::expr([
not::expr(is_l1_msg),
not::expr(is_padding_tx),
]),
);
```

chunk_txbytes_rlc

청크에 포함되는 모든 트랜잭션의 RLP 바이트를 RLC로 누적합니다:

```
// TxHashRLC 행에서
cb.condition(is_row_hash_rlc(meta), |cb| {
cb.condition(is_chunk_bytes, |cb| {
```

```
// chunk_txbytes_rlc 누적
cb.require_equal(
"chunk_txbytes_rlc accumulation",
chunk_txbytes_rlc(Rotation::next()),
chunk_txbytes_rlc * pow_of_rand + tx_hash_rlc,
);
// 길이 누적
cb.require_equal(
"chunk_txbytes_len_acc",
chunk_txbytes_len_acc(Rotation::next()),
chunk_txbytes_len_acc + tx_hash_length,
);
});
});
```

chunk_bytes_len

각 트랜잭션의 RLP 바이트 길이:

```
cb.condition(is_chunk_bytes, |cb| {
cb.require_equal(
"chunk_bytes_len = tx_hash_length",
chunk_bytes_len,
tx_hash_length,
```

```
);

});
```

청크 크기 제한

청크의 총 바이트 수는 제한이 있습니다:

```
const CHUNK_TXBYTES_BLOB_LIMIT: usize = (4096 * 31) - (2 + 45 * 4);

// = 126,896 바이트
```

이 제한을 초과하는 트랜잭션은 다음 청크로 이동합니다.

Witness 생성

Transaction 구조체

```
pub struct Transaction {

pub block_number: u64,

pub id: usize,

pub hash: H256,

pub tx_type: TxType,

pub nonce: u64,

pub gas: u64,

pub gas_price: Word,

pub max_fee_per_gas: Word,

pub max_priority_fee_per_gas: Word,
```

```
pub caller_address: Address,
pub callee_address: Option<Address>,
pub is_create: bool,
pub value: Word,
pub call_data: Vec<u8>,
pub call_data_length: usize,
pub call_data_gas_cost: u64,
pub access_list_gas_cost: u64,
pub tx_data_gas_cost: u64,
pub chain_id: u64,
pub rlp_unsigned: Vec<u8>, // 서명용 RLP
pub rlp_signed: Vec<u8>, // 트랜잭션 해시용 RLP
pub v: u64,
pub r: Word,
pub s: Word,
pub access_list: Option<AccessList>,
pub calls: Vec<Call>,
pub steps: Vec<ExecStep>,
}
```

고정 필드 할당

```
pub fn table_assignments_fixed<F: Field>(
```

```
&self,
challenges: Challenges<Value<F>>,
) -> Vec<[Value<F>; 5]> {
let  tx_hash_be_bytes = keccak256(&self.rlp_signed);
let  tx_sign_hash_be_bytes = keccak256(&self.rlp_unsigned);
vec![
// Nonce
[tx_id, TxFieldTag::Nonce, 0, nonce, 0],
// GasPrice (RLC)
[tx_id, TxFieldTag::GasPrice, 0,
rlc(&gas_price.to_le_bytes(), evm_word_challenge), 0],
// Gas
[tx_id, TxFieldTag::Gas, 0, gas, 0],
// CallerAddress
[tx_id, TxFieldTag::CallerAddress, 0,
caller_address.to_scalar(), 0],
// CalleeAddress
[tx_id, TxFieldTag::CalleeAddress, 0,
callee_address.unwrap_or(Address::zero()).to_scalar(), 0],
// IsCreate
[tx_id, TxFieldTag::IsCreate, 0, is_create  as u64, 0],
// Value (RLC)
[tx_id, TxFieldTag::Value, 0,
```

```
rlc(&value.to_le_bytes(), evm_word_challenge), 0],
// CallDataRLC
[tx_id, TxFieldTag::CallDataRLC, 0,
rlc_be_bytes(&call_data, keccak_input_challenge), 0],
// CallDataLength
[tx_id, TxFieldTag::CallDataLength, 0, call_data_length, 0],
// CallDataGasCost
[tx_id, TxFieldTag::CallDataGasCost, 0, call_data_gas_cost, 0],
// TxDataGasCost
[tx_id, TxFieldTag::TxDataGasCost, 0, tx_data_gas_cost, 0],
// ChainID
[tx_id, TxFieldTag::ChainID, 0, chain_id, 0],
// SigV
[tx_id, TxFieldTag::SigV, 0, v, 0],
// SigR (RLC)
[tx_id, TxFieldTag::SigR, 0,
rlc_be_bytes(&r.to_be_bytes(), evm_word_challenge), 0],
// SigS (RLC)
[tx_id, TxFieldTag::SigS, 0,
rlc_be_bytes(&s.to_be_bytes(), evm_word_challenge), 0],
// TxSignLength
[tx_id, TxFieldTag::TxSignLength, 0, rlp_unsigned.len(), 0],
// TxSignRLC
```

```
[tx_id, TxFieldTag::TxSignRLC, 0,
rlc_be_bytes(&rlp_unsigned, keccak_input_challenge), 0],
// TxSignHash
[tx_id, TxFieldTag::TxSignHash, 0,
rlc_be_bytes(&tx_sign_hash_be_bytes, evm_word_challenge), 0],
// TxHashLength
[tx_id, TxFieldTag::TxHashLength, 0, rlp_signed.len(), 0],
// TxHashRLC
[tx_id, TxFieldTag::TxHashRLC, 0,
rlc_be_bytes(&rlp_signed, keccak_input_challenge), 0],
// TxHash
[tx_id, TxFieldTag::TxHash, 0,
rlc_be_bytes(&tx_hash_be_bytes, evm_word_challenge), 0],
// TxType
[tx_id, TxFieldTag::TxType, 0, tx_type  as u64, 0],
// AccessListAddressesLen
[tx_id, TxFieldTag::AccessListAddressesLen, 0,
access_list_address_size, 0],
// AccessListStorageKeysLen
[tx_id, TxFieldTag::AccessListStorageKeysLen, 0,
access_list_storage_key_size, 0],
// AccessListRLC
[tx_id, TxFieldTag::AccessListRLC, 0,
```

```
rlc_be_bytes(&access_list.rlp_bytes(), keccak_input_challenge), 0],
// MaxFeePerGas (RLC)
[tx_id, TxFieldTag::MaxFeePerGas, 0,
rlc(&max_fee_per_gas.to_le_bytes(), evm_word_challenge), 0],
// MaxPriorityFeePerGas (RLC)
[tx_id, TxFieldTag::MaxPriorityFeePerGas, 0,
rlc(&max_priority_fee_per_gas.to_le_bytes(), evm_word_challenge), 0],
// BlockNumber
[tx_id, TxFieldTag::BlockNumber, 0, block_number, 0],
]
}
```

동적 필드 할당

CallData

```
pub fn table_assignments_dyn<F: Field>(
&self,
_challenges: Challenges<Value<F>>,
) -> Vec<[Value<F>; 5]> {
self.call_data
.iter()
.enumerate()
.map(|(idx, byte)| {
```

```
[
Value::known(F::from(self.id as u64)),
Value::known(F::from(TxFieldTag::CallData as u64)),
Value::known(F::from(idx  as u64)),
Value::known(F::from(*byte  as u64)),
Value::known(F::zero()),
]
})
.collect()
}
```

Access List

```
pub  fn  table_assignments_access_list_dyn<F: Field>(
&self,
challenges: Challenges<Value<F>>,
) -> Vec<[Value<F>; 5]> {
let  mut  assignments: Vec<[Value<F>; 5]> = vec![];
if  let Some(access_list) = &self.access_list {
let  mut  sks_acc: usize = 0;
for (al_idx, al) in  access_list.0.iter().enumerate() {
// AccessListAddress
assignments.push([
```

```
Value::known(F::from(self.id as u64)),
Value::known(F::from(TxFieldTag::AccessListAddress as u64)),
Value::known(F::from((al_idx + 1) as u64)),
Value::known(al.address.to_scalar().unwrap()),
Value::known(al.address.to_scalar().unwrap()),
]);
// AccessListStorageKey들
for  sk  in  al.storage_keys.iter() {
sks_acc += 1;
assignments.push([
Value::known(F::from(self.id as u64)),
Value::known(F::from(TxFieldTag::AccessListStorageKey as u64)),
Value::known(F::from(sks_acc  as u64)),
rlc_be_bytes(&sk.to_fixed_bytes(), challenges.evm_word()),
Value::known(al.address.to_scalar().unwrap()),
]);
}
}
}
assignments
}
```

RLP Witness 생성

```
pub fn gen_rlp_witness<F: Field>(
&self,
is_hash: bool,
challenges: &Challenges<Value<F>>,
) -> Vec<RlpFsmWitnessRow<F>> {
let (rlp_bytes, format) = if is_hash {
(
self.rlp_signed.clone(),
match self.tx_type {
TxType::Eip155 => TxHashEip155,
TxType::PreEip155 => TxHashPreEip155,
TxType::Eip1559 => TxHashEip1559,
TxType::L1Msg => L1MsgHash,
TxType::Eip2930 => TxHashEip2930,
},
)
} else {
(
self.rlp_unsigned.clone(),
match self.tx_type {
TxType::Eip155 => TxSignEip155,
TxType::PreEip155 => TxSignPreEip155,
```

```
TxType::Eip1559 => TxSignEip1559,

TxType::Eip2930 => TxSignEip2930,

_ => unreachable!(),

},

)

};

// RLP FSM을 사용하여 witness 생성

let mut witness = vec![];

let rom_table = format.rom_table_rows();

// ... RLP FSM 처리 로직

witness

}
```

패딩 트랜잭션

회로 크기가 실제 트랜잭션 수보다 큰 경우, 나머지 행은 패딩 트랜잭션으로 채워집니다.

패딩 트랜잭션 특징

```
pub fn dummy(chain_id: u64) -> Self {

let (dummy_tx, dummy_sig) = get_dummy_tx();

let dummy_tx_hash = get_dummy_tx_hash();

Self {

block_number: 0,
```

```
id: 0,
caller_address: Address::zero(), // 패딩 표시
callee_address: Some(Address::zero()),
is_create: false,
chain_id,
tx_data_gas_cost: tx_data_gas_cost(&rlp_signed),
v: dummy_sig.v,
r: dummy_sig.r,
s: dummy_sig.s,
rlp_signed,
rlp_unsigned,
hash: dummy_tx_hash,
tx_type: PreEip155,
..Default::default()
}
}
```

패딩 검증

```
// CallerAddress가 0이면 패딩 트랜잭션
cb.condition(is_caller_address(meta), |cb| {
cb.require_equal(
"is_padding_tx = (caller_address == 0)",
```

```
is_padding_tx,
value_is_zero.expr(),
);
});


// 패딩 트랜잭션은 서명 검증 건너뜀
cb.condition(is_padding_tx, |cb| {
// 서명 검증 비활성화
});
```

제약 조건 요약

1. Tag 전환 제약

```
// TxRomTable 룩업
meta.lookup_any("tx_table_tag_transition", |meta| {
let  cond = and::expr([
q_enable,
q_enable(Rotation::next()),
not::expr(q_first(Rotation::next())),
]);
vec![
(cond * tag, tx_rom_table.tag),
```

```
(cond * tag_next, tx_rom_table.tag_next),

(cond * tx_id_unchanged.is_equal_expression, tx_rom_table.is_tx_id_unchanged),

(cond * is_final, tx_rom_table.is_final),

(cond * q_dynamic_first(Rotation::next()), tx_rom_table.is_next_dynamic_first),

]

});
```

2. 트랜잭션 타입 제약

```
// 지원되는 타입만 허용

cb.require_in_set(

"tx_type supported",

tx_type,

vec![

PreEip155.expr(),

Eip155.expr(),

L1Msg.expr(),

Eip2930.expr(),

Eip1559.expr(),

],

);


// TxType 태그에서 tx_type과 value 일치
```

```
cb.condition(is_tx_type(meta), |cb| {
cb.require_equal(
"tx_type == value",
tx_type,
tx_table.value,
);
});
```

3. RLP 태그 제약

```
// 각 TxFieldTag에 대응하는 RlpTag 검증
cb.require_equal(
"associated rlp_tag",
rlp_tag,
rlp_tag_map.into_iter().fold(0.expr(), |acc, (expr, tag)| {
acc + usize::from(tag).expr() * expr
}),
);
```

4. is_none 제약

```
// is_none이면 value == 0
cb.condition(is_none_expr, |cb| {
```

```
cb.require_equal("value == 0", tx_table.value, 0.expr());
});

// CallDataRLC가 none이면 CallDataLength와 CallDataGasCost도 0
cb.condition(and::expr([is_data_rlc(meta), is_none_expr]), |cb| {
cb.require_equal("CallDataLength == 0",
tx_table.value(Rotation::next()), 0.expr());
cb.require_equal("CallDataGasCost == 0",
tx_table.value(Rotation(2)), 0.expr());
});
```

5. IsCreate 제약

```
// IsCreate == (CalleeAddress is none)
cb.condition(is_to(meta), |cb| {
cb.require_equal(
"is_create == is_none",
tx_table.value(Rotation::next()), // IsCreate
is_none,
);
});
```

6. CallData 제약

```
// CallDataLength가 0이 아니면 동적 섹션에 CallData 존재
cb.condition(and::expr([is_data_length(meta), not::expr(value_is_zero.expr())]),
// 동적 섹션 룩업 활성화
});


// CallData의 마지막 바이트에서 is_final == 1
cb.condition(and::expr([is_calldata(meta), is_final(meta)]), |cb| {
// section_rlc 검증
});
```

7. Access List 제약

```
// AccessListAddressesLen == 0이면 AccessListStorageKeysLen == 0
cb.condition(and::expr([is_access_list_addresses_len(meta), is_none]), |cb| {
cb.require_equal("AccessListStorageKeysLen == 0",
tx_table.value(Rotation::next()), 0.expr());
cb.require_equal("AccessListRLC == 0",
tx_table.value(Rotation(2)), 0.expr());
});


// Access List 인덱스 연속성
cb.condition(is_access_list_address(meta), |cb| {
```

```
cb.require_equal("al_idx increments",
al_idx, al_idx_prev + 1.expr());
});
```

8. 서명 검증 제약

```
// CallerAddress == sv_address (패딩 tx 제외)
cb.condition(and::expr([is_caller_address(meta), not::expr(value_is_zero.expr())
cb.require_equal("caller_address == sv_address",
tx_table.value, sv_address);
});
```

9. L1Msg 제약

```
// L1Msg는 TxDataGasCost == 0
cb.condition(and::expr([is_l1_msg(meta), is_tx_gas_cost(meta)]), |cb| {
cb.require_zero("tx_gas_cost == 0", tx_table.value);
});

// L1Msg는 서명 없음
cb.condition(is_l1_msg(meta), |cb| {
cb.require_zero("sig_v == 0", sig_v);
cb.require_zero("sig_r == 0", sig_r);
```

```
cb.require_zero("sig_s == 0", sig_s);

});
```

최적화 기법

1. RLC를 통한 데이터 압축

큰 값들(주소, Word 등)을 단일 필드 원소로 압축:

```
// 256비트 Word를 RLC로 압축

value_rlc = b[0] + b[1]*r + b[2]*r² + ... + b[31]*r³¹


// 여기서 r은 evm_word challenge
```

2. 도수 감소용 Boolean 컬럼

복잡한 표현식을 미리 계산하여 별도 컬럼에 저장:

```
// is_calldata, is_l1_msg, is_eip2930, is_eip1559 등

// 이를 통해 제약 조건의 차수(degree)를 낮춤
```

3. IsZero 가젯 재사용

0 검증이 필요한 여러 곳에서 IsZero 가젯을 재사용:

```
tx_id_is_zero: IsZeroConfig<F>, // tx_id == 0
```

```
value_is_zero: IsZeroConfig<F>, // value == 0
```

4. TxRomTable을 통한 Tag 전환 검증

고정 테이블을 사용하여 tag 전환 규칙을 효율적으로 검증:

```
// 모든 가능한 전환 시나리오를 미리 정의
// 런타임에 룩업만 수행
```

5. 조건부 룩업

필요한 경우에만 룩업을 활성화하여 비용 절감:

```
// CallDataLength > 0일 때만 CallData 룩업
let  enable = and::expr([
is_data_length(meta),
not::expr(value_is_zero.expr()),
]);
```

6. 누적 계산

반복적인 계산을 누적 방식으로 처리:

```
// CallData 가스 비용 누적
calldata_gas_cost_acc = calldata_gas_cost_acc_prev + gas_cost;

// Chunk bytes RLC 누적
```

```
chunk_txbytes_rlc = chunk_txbytes_rlc_prev * challenge + tx_hash_rlc;
```

테이블 연동

Tx Table

다른 회로(특히 EVM Circuit)는 Tx Table을 통해 트랜잭션 정보를 조회합니다.

테이블 스키마:

```
pub struct TxTable {
pub q_enable: Column<Fixed>,
pub tx_id: Column<Advice>,
pub tag: Column<Advice>,
pub index: Column<Advice>,
pub value: Column<Advice>,
pub access_list_address: Column<Advice>,
pub chunk_txbytes_hash_rlc: Column<Advice>,
}
```

사용 예시 (EVM Circuit에서):

```
// 트랜잭션의 CallerAddress 조회
lookup(
tx_id,
TxFieldTag::CallerAddress,
```

```
0, // index
caller_address, // value
)


// CallData 바이트 조회
lookup(
tx_id,
TxFieldTag::CallData,
byte_index,
byte_value,
)
```

Sig Table

Sig Circuit과 연동하여 서명을 검증합니다.

룩업 시점: ChainID 행 (각 트랜잭션당 1회)

룩업 데이터:

- 입력: tx_id, sig_v, sig_r, sig_s, tx_sign_hash
- 출력: sv_address (복구된 주소)

RLP Table

RLP Circuit과 연동하여 RLP 인코딩을 검증합니다.

룩업 시점:

- 서명용: 서명에 포함되는 각 필드

- 해시용: 해시에 포함되는 각 필드
- L1Msg: L1 메시지 해시에 포함되는 각 필드

룩업 데이터:

- tx_id, rlp_tag, is_none, value_length, value_rlc

Keccak Table

Keccak Circuit과 연동하여 해시를 검증합니다.

룩업 시점:

- TxSignLength 행: 서명 해시 검증 (L2 tx)
- TxHashLength 행: 트랜잭션 해시 검증 (L1Msg)

룩업 데이터:

- 입력: rlc, length
- 출력: hash

Block Table

Block Circuit과 연동하여 블록 정보를 검증합니다.

룩업 데이터:

- CumNumTxs: 누적 트랜잭션 수
- NumTxs: 블록당 트랜잭션 수
- NumAllTxs: 총 트랜잭션 수 (L1 + L2)

구현 예시

간단한 트랜잭션 검증

```
// PreEip155 트랜잭션

let  tx = Transaction {

id: 1,

tx_type: TxType::PreEip155,

nonce: 5,

gas_price: Word::from(20_000_000_000u64), // 20 Gwei

gas: 21000,

caller_address: Address::from_str("0x1234...").unwrap(),

callee_address: Some(Address::from_str("0x5678...").unwrap()),

is_create: false,

value: Word::from(1_000_000_000_000_000_000u64), // 1 ETH

call_data: vec![],

call_data_length: 0,

call_data_gas_cost: 0,

chain_id: 1,

v: 27,

r: Word::from_str("0xabcd...").unwrap(),

s: Word::from_str("0xef01...").unwrap(),

..Default::default()

};



// Tx Table 할당 (고정 섹션만, 28행)
```

```
let fixed_assignments = tx.table_assignments_fixed(&challenges);

// CallData가 없으므로 동적 섹션 없음
```

EIP-1559 트랜잭션 with Access List

```
let access_list = AccessList(vec![

AccessListItem {

address: Address::from_str("0xaaaa...").unwrap(),

storage_keys: vec![

H256::from_str("0x0000...0001").unwrap(),

H256::from_str("0x0000...0002").unwrap(),

],

},

AccessListItem {

address: Address::from_str("0xbbbb...").unwrap(),

storage_keys: vec![

H256::from_str("0x0000...0003").unwrap(),

],

},

]);


let tx = Transaction {

id: 2,
```

```
tx_type: TxType::Eip1559,
max_fee_per_gas: Word::from(100_000_000_000u64), // 100 Gwei
max_priority_fee_per_gas: Word::from(2_000_000_000u64), // 2 Gwei
access_list: Some(access_list),
call_data: vec![0x60, 0x60, 0x60, 0x40], // 4 bytes
call_data_length: 4,
call_data_gas_cost: 64, // 4 * 16
..Default::default()
};


// Tx Table 할당
let  fixed_assignments = tx.table_assignments_fixed(&challenges); // 28행
let  calldata_assignments = tx.table_assignments_dyn(&challenges); // 4행
let  access_list_assignments = tx.table_assignments_access_list_dyn(&challenges)
// 총 37행
```

L1 메시지

```
let  l1_msg = Transaction {
id: 3,
tx_type: TxType::L1Msg,
nonce: 10, // queue_index
gas: 100000,
```

```
caller_address: Address::from_str("0xL1...").unwrap(),

callee_address: Some(Address::from_str("0xL2...").unwrap()),

value: Word::from(500_000_000_000_000_000u64), // 0.5 ETH

call_data: vec![0xa9, 0x05, 0x9c, 0xbb], // function selector

tx_data_gas_cost: 0, // L1에서 이미 지불

v: 0, // 서명 없음

r: Word::zero(),

s: Word::zero(),

..Default::default()

};



// L1Msg는 서명 검증 건너뜀

// 해시는 Keccak Table에서 직접 검증
```

관련 문서

- Sig Circuit: ECDSA 서명 검증
- RLP Circuit: RLP 인코딩 검증
- Keccak Circuit: 해시 계산
- EVM Circuit: Tx Table 사용
- Tx Table: 테이블 스키마 상세
- RLC 기법: Random Linear Combination 설명

참고 자료

주요 파일:

- `zkevm-circuits/src/tx_circuit.rs` : 회로 구현
- `zkevm-circuits/src/witness/tx.rs` : Witness 생성
- `zkevm-circuits/src/table.rs` : TxTable 및 TxFieldTag 정의
- `eth-types/src/geth_types.rs` : TxType 정의

핵심 함수:

- `TxCircuitConfig::new()` : 회로 설정
- `Transaction::table_assignments_fixed()` : 고정 필드 할당
- `Transaction::table_assignments_dyn()` : CallData 할당
- `Transaction::table_assignments_access_list_dyn()` : Access List 할당
- `Transaction::sign_data()` : 서명 데이터 생성
- `Transaction::gen_rlp_witness()` : RLP witness 생성
- `recover_pk2()` : 공개키 복구
- `tx_data_gas_cost()` : RLP 가스 비용 계산
- `tx_access_list_gas_cost()` : Access List 가스 비용 계산

핵심 구조체:

- `TxCircuitConfig` : 회로 설정
- `Transaction` : 트랜잭션 데이터
- `TxRomTable` : Tag 전환 규칙
- `SignData` : 서명 데이터
- `AccessList` : Access List 구조